

Projeto Calculadora — Módulo 03

Turma: LionsDev

Tópicos: Estruturas de Controle, Loops, Funções, Validação de Entrada, Menu Interativo, Acumulador de Resultado.

1. Visão Geral

Neste módulo você constrói o primeiro projeto com lógica de verdade do curso: uma calculadora de terminal em Node.js.

Objetivo de aula: ensinar, na prática, estruturas de controle de fluxo (`if` / `switch`), loops (`while`), funções e validação de entrada de dados, usando a calculadora como caso de uso.

O programa deve:

- Oferecer um menu de operações matemáticas;
- Ler números do usuário com `prompt` ;
- Validar entradas inválidas (ex.: texto onde se espera número);
- Manter um resultado acumulado entre operações;
- Permitir encerrar o programa de forma explícita.

2. Por Que Definir Requisitos Antes de Codar?

Sem requisitos, "fazer uma calculadora" é tão vago quanto dizer "**vamos construir um carro**": não sabemos se é popular, esportivo, elétrico, quantas portas, orçamento, etc.

Requisitos reduzem retrabalho: você sabe quando terminou e o que testar.

3. Escopo — Qual Calculadora Vamos Criar?

Para este projeto, o escopo inclui:

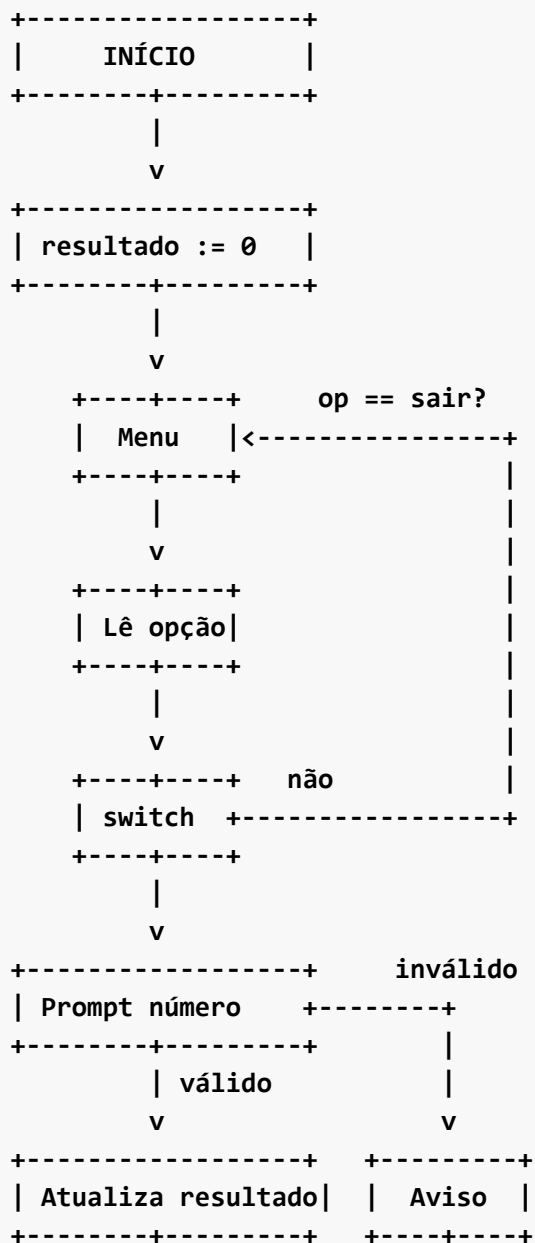
- Adição
- Subtração
- Multiplicação
- Divisão
- Porcentagem (em relação ao valor acumulado)
- Exibir o resultado atual ("Ver Resultado")
- Sair do programa

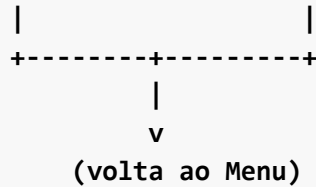
O que fica **fora do escopo** (a menos que você decida estender):

- Operações científicas (seno, log, potência com expoente variável);
- Histórico de todas as operações em arquivo;
- Interface gráfica (GUI).

4. Fluxo de Execução

1. Inicializar acumuladores (ex.: resultado em zero).
2. Repetir até o usuário pedir para sair:
 - Mostrar menu de operações no terminal.
 - Ler a opção escolhida (**prompt**).
 - Conforme a opção: pedir número(s), validar, chamar a função da operação e atualizar o acumulado; ou mostrar o resultado; ou encerrar.
 - Voltar ao menu (continua o loop).
3. Mensagem de encerramento.





- **Interatividade:** a cada volta do loop o usuário vê o menu e decide.
- **Menu:** centraliza as escolhas numéricas (1..7) mapeadas no `switch`.
- **Múltiplas operações:** o `while` mantém o programa vivo; o acumulador guarda o estado entre uma operação e outra.
- **Persistência do resultado:** neste projeto é "em memória" apenas (variável que sobrevive enquanto o processo Node está rodando).

5. Validação de Entrada

- **Robustez:** evita `NaN` silencioso ou comportamento estranho em cálculos.
- **Experiência do usuário:** mensagens claras ("digite um número") em vez de travar.
- **Manutenção:** quando algo dá errado, fica mais fácil saber se foi a entrada.

Na prática do `prompt`, tudo chega como string; por isso o padrão do projeto é converter para número e depois validar:

```

num = Number(prompt("..."));
if (isNaN(num)) {
  console.log("Da próxima vez digite um número correto!");
} else {
  // ... chama função matemática ...
}
  
```

6. Escopo Mínimo Obrigatório

Critérios para considerar o projeto completo:

- O programa exibe um menu numérico e aceita apenas opções válidas ou informa erro para opção inválida.
- Para cada operação que pede número, se o usuário digitar algo que não converte para número útil, o sistema avisa e não corrompe o estado.
- O laço principal só termina quando o usuário escolhe sair.
- Existe forma de ver o resultado acumulado sem precisar aplicar outra operação.
- Divisão trata divisão por zero com mensagem clara.
- Cada operação matemática está isolada em sua própria função (`somar`, `subtrair`, `multiplicar`, `dividir`, `porcentagem`).
- O resultado fica acumulado entre operações, sem resetar a cada chamada.

7. Indo Além (Opcional)

O escopo mínimo da seção 6 é obrigatório e precisa funcionar antes de qualquer extra. A partir daí, quem quiser pode acrescentar operações novas ao menu, seguindo o mesmo padrão do projeto: função isolada, validação de entrada, caso tratado no `switch`.

Exemplos de operações extras:

- **Teorema de Pitágoras:** hipotenusa a partir de dois catetos (`Math.sqrt(a**2 + b**2)`).
- **Potenciação e raiz quadrada:** `Math.pow` , `Math.sqrt` .
- **Limite de uma função:** aproximação numérica, avaliando a função com `x` cada vez mais próximo do ponto desejado.
- **Derivada de uma função:** aproximação por diferença finita (`(f(x + h) - f(x)) / h` , com `h` bem pequeno).
- Outras ideias por conta do aluno: média, fatorial, conversão de unidades, etc.

Essas extensões são opcionais e não substituem o escopo mínimo obrigatório.

Próximo passo no curso (código modular):

`modulo05/conteudo/modulos.js` → `modulo05/exercicios_slides/calculadora/`
(cada operação em arquivo separado)